The Journal of Open Source Software

Minimap2-rs: Rust bindings for Minimap2

Joseph Guhlin [1]

1 Genomics Aotearoa, Department of Biochemistry, University of Otago, Dunedin, New Zealand

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: William Gearty

Reviewers:

- @sstadick
- @samuelortion

Submitted: 06 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0)

Summary

Long-read sequence alignment underpins modern genomic research, enabling highly contiguous de novo assemblies, structural variant detection, RNA isoform exploration, and many other applications. Minimap2, the long-read sequence aligner, sits at the center of these developments, providing fast alignment for long and noisy reads. The Rust programming language has seen rapid adoption in the scientific community, offering memory safety, concurrency, high performance, and simplified dependency management. Here, we present Minimap2-rs, a collection of Rust crates providing a Rust-centric interface to Minimap2, lowering the barrier to using this aligner in Rust-based applications.

The top-level Minimap2 crate offers an opinionated, flexible, idiomatic API that manages memory and simplifies calling C functions via the Rust foreign function interface (FFI). A companion Python library, minimappers2, provides multi-threading and returns high-performance dataframes. The minimap2-sys crate provides direct, low-level bindings to Minimap2 for specialized use cases. This modular design facilitates rapid prototyping and production-ready pipelines. Continuous integration across x86_64 and arm64 architectures ensures reliability on Linux, macOS, and Android, with tested support for musl-based portability. Minimap2-rs has been adopted by multiple tools for genome size estimation, tandem repeat genotyping, and transcriptome analysis. Minimap2-rs offers a stable, open-source foundation for building Rust-powered bioinformatics solutions while preserving Minimap2's speed and flexibility.

Statement of Need

Long-read sequence alignment has changed the foundation of recent genomic advances. Thanks to long-read sequence alignment, advances such as the telomere-to-telomere Human genome assembly (Nurk et al., 2022) were completed, the cost to assemble a *de novo* genome has decreased significantly, contiguity has increased (Cheng et al., 2021; Kolmogorov et al., 2019; Koren & Phillippy, 2015; Murigneux et al., 2020; Reis et al., 2023), and it is now used for RNA-based applications (Jain et al., 2022). The primary algorithm supporting nearly all of these advancements is Minimap2, which aligns long-read sequences to large references while accounting for the high error rate in long reads (Li, 2018, 2021). Minimap2 can also handle spliced, short-read, and assembly-to-assembly alignments.

The Rust programming language has seen significant adoption, including amongst scientists (Bugden & Alahmar, 2022; Matsakis & Klock, 2014; Perkel, 2020). Rust brings memory safety, strong ownership, and performance to applications and libraries, with consistent build tooling (Bugden & Alahmar, 2022). This ultimately enables highly portable applications with strong concurrency idioms, encouraging high-performance by default. With the growing adoption of Rust, there are now several tools and libraries in Rust for bioinformatics (Buffalo, 2024; Chan, 2024; Huey & Abdennur, 2024; Köster, 2016).

DRAFT

State of the Field

Minimap2 is a command-line tool and library, and has an official Python binding, Mappy. Using Minimap2 directly in Rust requires either foreign function interface (FFI) calls or running Minimap2 as a subprocess, which may not be ideal for situations that require custom workflows. Minimap2-rs provides FFI bindings to the Minimap2 library, exposing the underlying API directly to Rust and supporting an idiomatic Rust API that works with the Rust ownership model, simplifying the interface with Minimap2 from Rust. Minimap2-rs is open source and publicly available through crates.io, the standard library registry for Rust, and GitHub. Rammap (Wang & Li, 2026) is a full reimplementation of minimap2 in pure Rust, aiming for output identical to minimap2. Minimap2-rs binds the official C implementation, so downstream tools receive new upstream minimap2 releases, presets, and options, and alignments are identical to the minimap2 command-line tool of the same release, given the same options. Each crate version pins a specific minimap2 release (e.g., 0.1.31+minimap2.2.31), allowing for version-specific reproducibility. However, minimap2-rs requires a C toolchain to build and the FFI boundary relies on unsafe code, whereas the pure-Rust reimplementation avoids both and permits modification of internal components. Rammap is API-compatible with minimap2-rs, so switching between the two should be straightforward.

Software Design

Minimap2 uses single-instruction multiple-data (SIMD) CPU features to achieve higher performance and better parallelization, enabling it to handle large datasets and long reads. Minimap2-rs consists of three libraries (called 'crates' in Rust): Minimap2, minimappers2, and minimap2-sys. The idiomatic entry point is the Minimap2 Rust crate (further referred to as minimap2-rs), which provides an opinionated, memory-safe API for working with Minimap2 functions via the foreign function interface (FFI) and returning results in Rust data structures (structs), automatically converting base types between C and Rust. Minimappers2 is a Python binding wrapping minimap2-rs, and an alternative to minimap2's own Mappy, providing built-in multi-threading and returning results as high-performance, memory-efficient data frames. In Rust, there is a convention of using a separate crate to wrap the C library, and that convention is followed here: Minimap2-sys wraps the C library and provides a direct interface to the C functions in Rust via unsafe code (Matsakis & Klock, 2014). A functional fourth tool, fakeminimap2, is available and serves as a coding exemplar of two common multithreading approaches.

Minimap2-rs is the primary interface for using Minimap2 in downstream Rust applications. An aligner struct is created, and presets, mapping, and indexing options are configured within it. All presets from the Minimap2 command-line software (e.g., map-ont, map-pb, map-hifi, and asm20), as well as low-level access to additional index and mapping options in the Minimap2 C library, are available. The crate supports optional features (see 1). Minimap2 is built using Rust's tooling; thus, adding this library only requires adding it as a dependency. The code repository provides two examples of multithreading implementation with minimap2-rs: a multi-producer single-consumer (MPSC) channel and parallelization using the popular Rayon crate.

Table 1: Rust crate-level features available for `minimap2-rs`. []{} Rust features enable conditional compilation and reduce dependencies when not needed. 1 Enabled by default.

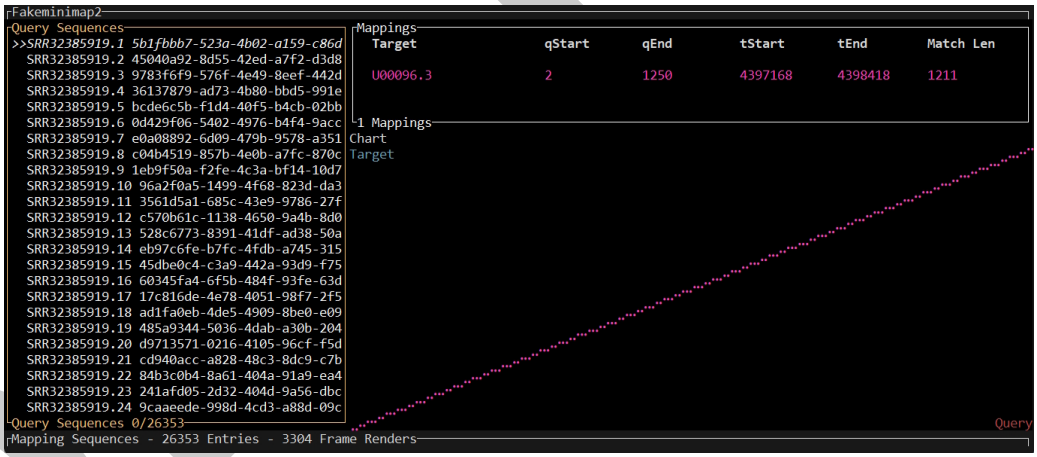
Feature	Description
`map-file` 1	Adds a convenience function to parse and map a FASTA/Q file to a provided index or reference.
`htslib`	Supports returning results as HTS records.
`curl`	Enables curl support for htslib.

DRAFT

Feature	Description
`simde`	Enables the SIMD Everywhere library in minimap2 compilation.
`zlib-ng`	Uses zlib-ng in the minimap2 index for faster reading of compressed files.
`static`	Builds minimap2-rs as a static library in Rust.
`sse2only`	Builds minimap2 with only SSE2 support.

Minimap2-sys is the low-level, direct FFI interface to the Minimap2 library, containing most unsafe function calls, with automatically generated bindings from the bindgen crate, and minor improvements to prevent memory leaks in downstream applications. The majority of the code here is in the build.rs file, which builds the Minimap2 C library, enables architecture-specific features, and allows Minimap2 to link to Rust. Compatible operating systems for minimap2-rs are Linux and macOS, on supported architectures x86_64 and aarch64/arm64. Maximum portability of downstream binaries can be achieved by compiling for musl, an alternative libc implementation. These are tested with continuous integration to prevent any regressions or loss of system support. All features are tested at this time. Experimental Android support is also tested on aarch64 and x86_64.

Fakeminimap2 provides in-depth examples of multi-threading using both MPSC channels and Rayon, as well as a terminal user interface (TUI) that supports interactive tables and visualizations of alignments (see 4), and handles mouse interaction events in supported terminals.



Research Impact Statement

Minimap2-rs is used by multiple tools for sequence cleaning (De Coster & Rademakers, 2023), tandem-repeat genotyping (De Coster et al., 2024), long-read transcriptome quantification (previously, prior to 0.10) (Jousheghani & Patro, 2024), long-read-based genome size estimation (Hall & Coin, 2024), pharmacogenomic diplotyping (Holt et al., 2024), and genotyping for large-unit variable number tandem repeats (Chen et al., 2026). As of this publication, there are 17 additional, non-automated contributors to the repo.

AI Usage Disclosure

This library was primarily written manually, with generative AI used only for some recent bugs and for exploring features in branches not part of the release. All generated code was manually reviewed and tested against the existing test suite, which also verifies correctness against the official minimap2 binary. Generative coding AI was also used to expand the C to Rust mapping options and explanations found in the README.

DRAFT

Acknowledgements

The author wishes to thank Wouter De Coster for kicking off this project with a tweet and Heng Li for creating Minimap2 and supporting this library. The author also thanks the numerous contributors to GitHub, both for feature requests, bug reports, and especially for pull requests. Finally, a thank you to Peter Dearden for helping with manuscript prep and proofreading.

Conflict of Interest

None declared.

Funding

This work was funded by a grant from Genomics Aotearoa (High Quality Genomes II), which is funded by the New Zealand Ministry of Business, Innovation and Employment.

Data Availability

The software repository is available on GitHub at https://github.com/jguhlin/minimap2-rs, and all published versions of this crate are available at crates.io at https://crates.io/crates/minimap2. API documentation is available at https://docs.rs/minimap2. Minimappers2 is available through PyPI at https://pypi.org/project/minimappers2/. The software is available under the MIT or Apache 2.0 License, at the user's discretion. Fakeminimap2 data displayed used Nanopore reads from NCBI SRA SRR32385919 and the whole-genome sequence NCBI GenBank U00096.3.

Buffalo, V. (2024). GRanges: A rust library for genomic range data. *bioRxiv*, 2024–2005. https://doi.org/10.1101/2024.05.24.595786

Bugden, W., & Alahmar, A. (2022). Rust: The programming language for safety and performance. *arXiv Preprint arXiv:2206.05503*. https://doi.org/10.48550/arXiv.2206.05503

Chan, K. O. (2024). Next-generation bioinformatics: An ultrafast and user-friendly tool for phylogenomic data exploration. *Molecular Ecology Resources*, *24*(7), e13993. https://doi.org/10.22541/au.172328511.13448312/v1

Chen, X., Lemmers, R. J. L. F., Kronenberg, Z., Devaney, J. M., Noya, J., Berlyoung, A. S., Yusuff, S., Lynch, S., Nykamp, K., Lindy, A. S., Dolzhenko, E., Maarel, S. M. van der, & Eberle, M. A. (2026). Resolution of the D4Z4 repeat responsible for facioscapulohumeral muscular dystrophy with HiFi sequencing. *bioRxiv*. https://doi.org/10.64898/2026.04.10.717730

Cheng, H., Concepcion, G. T., Feng, X., Zhang, H., & Li, H. (2021). Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature Methods*, *18*(2), 170–175. https://doi.org/10.1038/s41592-020-01056-5

De Coster, W., Höijer, I., Bruggeman, I., D'Hert, S., Melin, M., Ameur, A., & Rademakers, R. (2024). Visualization and analysis of medically relevant tandem repeats in nanopore sequencing of control cohorts with pathSTR. *Genome Research*, *34*(11), 2074–2080. https://doi.org/10.1101/gr.279265.124

De Coster, W., & Rademakers, R. (2023). NanoPack2: Population-scale evaluation of long-read sequencing data. *Bioinformatics*, *39*(5), btad311. https://doi.org/10.1093/bioinformatics/btad311

DRAFT

Hall, M. B., & Coin, L. J. M. (2024). Genome size estimation from long read overlaps. *bioRxiv*, 2024.11.27.625777. https://doi.org/10.1101/2024.11.27.625777

Holt, J. M., Harting, J., Chen, X., Baker, D., Saunders, C. T., Kronenberg, Z., Gonzaludo, N., Yoo, B., Hudjashov, G., Jõeloo, M., Lawlor, J. M. J., Lim, W. K., Team, E. B. R., Jamuar, S. S., Cooper, G. M., Milani, L., Pastinen, T., & Eberle, M. A. (2024). StarPhase: Comprehensive phase-aware pharmacogenomic diplotyper for long-read sequencing data. *bioRxiv*. https://doi.org/10.1101/2024.12.10.627527

Huey, J. D., & Abdennur, N. (2024). Bigtools: A high-performance BigWig and BigBed library in rust. *Bioinformatics*, *40*(6). https://doi.org/10.1093/bioinformatics/btae350

Jain, M., Abu-Shumays, R., Olsen, H. E., & Akeson, M. (2022). Advances in nanopore direct RNA sequencing. *Nature Methods*, *19*(10), 1160–1164. https://doi.org/10.1038/s41592-022-01633-w

Jousheghani, Z. Z., & Patro, R. (2024). Oarfish: Enhanced probabilistic modeling leads to improved accuracy in long read transcriptome quantification. *bioRxiv*. https://doi.org/10.1101/2024.02.28.582591

Kolmogorov, M., Yuan, J., Lin, Y., & Pevzner, P. A. (2019). Assembly of long, error-prone reads using repeat graphs. *Nature Biotechnology*, *37*(5), 540–546. https://doi.org/10.1038/s41587-019-0072-8

Koren, S., & Phillippy, A. M. (2015). One chromosome, one contig: Complete microbial genomes from long-read sequencing and assembly. *Current Opinion in Microbiology*, *23*, 110–120. https://doi.org/10.1016/j.mib.2014.11.014

Köster, J. (2016). Rust-bio: A fast and safe bioinformatics library. *Bioinformatics*, *32*(3), 444–446. https://doi.org/10.1093/bioinformatics/btv573

Li, H. (2018). Minimap2: Pairwise alignment for nucleotide sequences. *Bioinformatics*, *34*(18), 3094–3100. https://doi.org/10.1093/bioinformatics/bty191

Li, H. (2021). New strategies to improve minimap2 alignment accuracy. *Bioinformatics*, *37*(23), 4572–4574. https://doi.org/10.1093/bioinformatics/btab705

Matsakis, N. D., & Klock, F. S. (2014). The rust language. *ACM SIGAda Ada Letters*, *34*(3), 103–104. https://doi.org/10.1145/2663171.2663188

Murigneux, V., Rai, S. K., Furtado, A., Bruxner, T. J., Tian, W., Harliwong, I., Wei, H., Yang, B., Ye, Q., Anderson, E., & others. (2020). Comparison of long-read methods for sequencing and assembly of a plant genome. *GigaScience*, *9*(12), giaa146. https://doi.org/10.1093/gigascience/giaa146

Nurk, S., Koren, S., Rhie, A., Rautiainen, M., Bzikadze, A. V., Mikheenko, A., Vollger, M. R., Altemose, N., Uralsky, L., Gershman, A., & others. (2022). The complete sequence of a human genome. *Science*, *376*(6588), 44–53. https://doi.org/10.1126/science.abj6987

Perkel, J. M. (2020). Why scientists are turning to rust. *Nature*, *588*, 185. https://doi.org/10.1038/d41586-020-03382-2

Reis, A. L. van der, Beckley, L. E., Olivar, M. P., & Jeffs, A. G. (2023). Nanopore short-read sequencing: A quick, cost-effective and accurate method for DNA metabarcoding. *Environmental DNA*, *5*(2), 282–296. https://doi.org/10.1002/edn3.374

Wang, J. R., & Li, H. (2026). Memory-safe high-performance sequence mapping with rammap. *bioRxiv*. https://doi.org/10.64898/2026.05.26.726289

DRAFT